OmahTI
Organisasi Mahasiswa Ahli Teknologi Informasi

Modul Backend #8

Transaksi & Quality



Gedung Fakultas MIPA UGM, Sekip Utara, Bulaksumur, Sinduandi, Mlati, Sleman, D.I Yogyakarta

omahti.mipa@ugm.ac.id

Visit us on
omahti.web.id

Table of Contents

1. Tujuan Pembelajaran 3
2. Transaksi Database & Konsep ACID 3
   I. Real World Examples 4
   II. Anomali Concurrency 4
   III. Isolation Level 5
   IV. Deadlock 5
   V. Idempotency 6
3. Implementasi DB Transaction 6
   I. Bad Practices 7
   II. Best Practices 7
4. Software Quality & Unit Testing 9
   I. Kenapa Perlu Testing? 9
   II. AAA Pattern 9
   III. Setup & Tools 9
   IV. Mocking 10
   V. Kenapa Harus Mocking? 10
5. Unit Test untuk Logic Transaksi 11
   I. Measuring Test Coverage 14
   II. Menjalankan Coverage 14

1. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, peserta diharapkan untuk:

1. Memahami kenapa operasi database yang saling berkaitan tidak boleh berjalan setengah jalan, terutama pada sistem yang melibatkan transfer data, pembayaran, voting, atau proses kritikal lainnya.
2. Memahami bagaimana Atomicity, Consistency, Isolation, dan Durability bekerja untuk menjaga integritas data pada aplikasi modern.
3. Memahami bahwa kode yang “berjalan” belum tentu aman untuk production, dan bahwa testing merupakan bagian fundamental dari proses engineering modern.

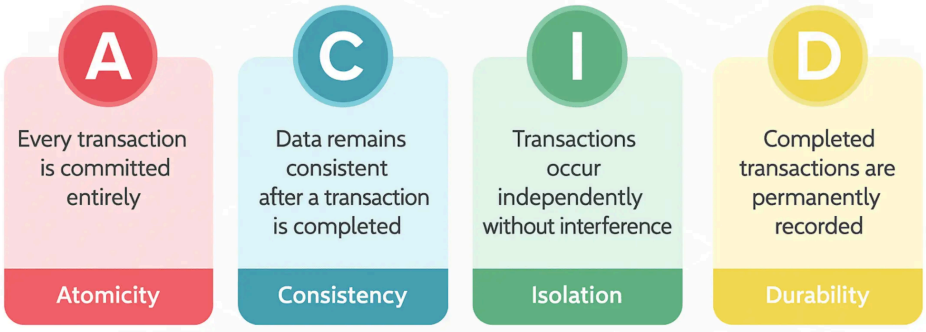
2. Transaksi Database & Konsep ACID

Apa itu transaksi database?

Di dunia nyata, aplikasi jarang hanya melakukan satu query tunggal. Seringkali, satu action dari user memicu serangkaian query beruntun.

Secara teknis, transaction adalah sekelompok operasi database yang diperlakukan sebagai satu unit kerja tunggal. Artinya: semua operasi di dalam kelompok itu harus berhasil semua, atau kalau ada satu aja yang gagal, semua operasi sebelumnya dibatalkan seolah-olah nggak pernah terjadi. Nggak ada cerita "setengah jadi".

Apa itu ACID?



ACID adalah garansi yang diberikan oleh *database relational* (seperti PostgreSQL atau MySQL) untuk setiap transaksi.

- **Atomicity**: *All or nothing.* Semua query sukses, atau semuanya batal.
- **Consistency**: Data selalu valid dan mematuhi aturan database (misal: saldo tidak boleh minus).
- **Isolation**: Transaksi yang berjalan secara bersamaan nggak saling mempengaruhi satu sama lain selama proses berlangsungnya
- **Durability**: Jika transaksi sukses (*commit*), data aman tersimpan meski *server* mati mendadak.

Real World Examples

- **Sistem E-Wallet:** Misal ada sebuah *request* transfer saldo. Ketika saldo pengirim dikurangi, server crash sebelum sempat menambahkan saldo ke penerima. Disini transaksi berperan untuk memastikan eksekusi beruntun tidak boleh sukses "setengah jalan". Jika langkah 2 gagal, maka langkah 1 harus dibatalkan (*rollback*)
- **Sistem Voting (Race Condition):** Bayangkan 1000 orang menekan tombol "Vote" di detik yang sama pada sebuah kontes. Tanpa transaksi (dan locking yang tepat), sistem bisa membaca jumlah vote yang sama secara paralel, lalu menambahkannya, sehingga dari 1000 klik, mungkin hanya tercatat 200 vote.

Anomali Concurrency

1. Dirty Read
   Dirty read adalah kondisi dimana transaksi membaca data “bohongan”, misal transaction A belum commit, tetapi sudah dibaca transaction B. Ternyata transaction A rollback..

2. Non-Repeatable Read
   Misal kita ngecek harga tiket pesawat, awalnya Rp 1 juta. Terus di refresh, harganya naik jadi Rp 1,2 juta karena orang lain beli tiket.

3. Phantom Read
   Misal kita melakukan sebuah query, dan meng-output 10 row. Tiba tiba transaction lain insert data baru. Query kedua kita sekarang meng-output 11 row. Data yang dilihat berbeda padahal masih dalam satu transaksi

Isolation Level

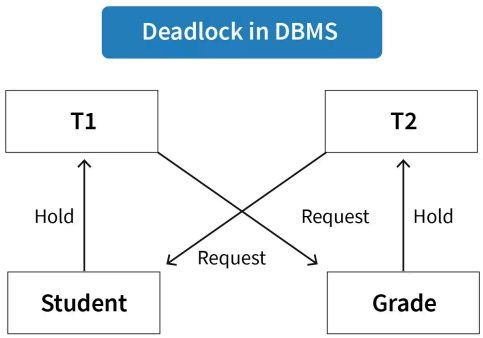
Ada beberapa level isolasi untuk menangani anomali concurrency tadi,

Isolation level	Penjelasan	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Bisa baca data yang belum commit	Diperbolehkan	Diperbolehkan	Diperbolehkan
Read Committed	Hanya baca data yang sudah commit	Dilarang	Diperbolehkan	Diperbolehkan
Repeatable Read	Hasil SELECT tetap konsisten selama transaction	Dilarang	Dilarang	Diperbolehkan
Serializable	Transaction seolah berjalan satu-satu	Dilarang	Dilarang	Dilarang

Di kebanyakan use case, default level di PostgreSQL (`Read Committed`) atau MySQL InnoDB (`Repeatable Read`) sudah cukup.

Deadlock

Deadlock adalah kondisi ketika dua transaction saling menunggu satu sama lain. Akibatnya, tidak ada yang bisa lanjut dan tidak ada yang selesai.



Untuk mengurangi atau mencegah deadlock pada basis data, dapat kita paksa kedua transaksi untuk selalu mengakses Tabel A terlebih dahulu, baru kemudian Tabel B. Dengan begitu, salah satu transaksi akan mengantri sejak awal.

Idempotency

Idempotency (idempotensi) adalah sebuah konsep di mana suatu operasi atau aksi bisa dilakukan berulang kali, tetapi memberikan hasil akhir yang tetap sama seperti saat pertama kali dijalankan.

Misal user meng-klik tombol “Bayar” berulang kali untuk suatu produk. Jika tidak idempoten, saldo user akan terpotong berkali-kali untuk produk yang sama. Jika idempoten, pembayaran kedua dan setelahnya akan diabaikan dan saldo tidak berkurang lagi.

Untuk mengimplementasi idempotency, ada 4 langkah utama:

1. Klien membuat token unik.
   Sebelum mengirim data, aplikasi di HP atau web pengguna membuat sebuah kode unik acak yang tidak mungkin kembar (biasanya menggunakan UUID atau GUID).

2. Kunci Dikirim ke Server
   Klien mengirimkan data transaksi *sekaligus* menyertakan kode unik tersebut di dalam komponen *Header* request API.

3. Server Memeriksa Kunci di Database/Cache
   Jika kunci belum ada, berarti ini permintaan pertama dan server akan memproses request.

4. Batas Waktu (TTL)
   Kunci unik idempotensi diberi *Time-To-Live (TTL)* agar kunci tersebut otomatis terhapus setelah 24 hingga 48 jam untuk menghemat memori

3. Implementasi DB Transaction

Di level kode, transaksi memiliki siklus hidup yang sederhana namun absolut:

1. **BEGIN:** Memberitahu *database* bahwa rangkaian operasi dimulai.
2. **COMMIT:** Menyimpan semua perubahan secara permanen jika tidak ada *error*.
3. **ROLLBACK:** Membatalkan seluruh perubahan jika terjadi *error* di tengah proses

.

Bad Practices

Jika kita tidak menggunakan transaksi, kegagalan pada query kedua akan membuat data menjadi tidak konsisten.

Contoh implementasi kode *unsafe* menggunakan SQL:

```go
func TransferNaifSQL(db *sql.DB, fromID, toID int, amount float64) error {
    // 1. Kurangi saldo pengirim
    _, err := db.Exec("UPDATE accounts SET balance = balance - $1 WHERE id = $2", amount, fromID)
    if err != nil {
        return err
    }

    // Jika server crash di sini, uang hilang

    // 2. Tambah saldo penerima
    _, err = db.Exec("UPDATE accounts SET balance = balance + $1 WHERE id = $2", amount, toID)
    return err
}
```

Contoh implementasi kode *unsafe* menggunakan ORM:

```go
func TransferNaifGorm(db *gorm.DB, fromID, toID int, amount float64) error {
    // 1. Kurangi saldo pengirim
    err := db.Model(&Account{}).Where("id = ?", fromID).
        UpdateColumn("balance", gorm.Expr("balance - ?", amount)).Error
    if err != nil {
        return err
    }

    // Sama rentannya jika crash di titik ini.

    // 2. Tambah saldo penerima
    return db.Model(&Account{}).Where("id = ?", toID).
        UpdateColumn("balance", gorm.Expr("balance + ?", amount)).Error
}
```

Best Practices

Pola terbaik mengharuskan kita mengontrol lifecycle transaksi. Jika ada *error* atau *panic* di tengah jalan, sistem harus otomatis melakukan *rollback.*

Contoh implementasi kode *best practice* menggunakan SQL:

```go
func TransferAmanSQL(db *sql.DB, fromID, toID int, amount float64) error {
    tx, err := db.Begin()
    if err != nil { return err }

    defer tx.Rollback() // Jaring pengaman otomatis

    if _, err := tx.Exec("UPDATE accounts SET balance = balance - $1 WHERE id = $2", amount, fromID); err != nil {
        return err
    }

    if _, err := tx.Exec("UPDATE accounts SET balance = balance + $1 WHERE id = $2", amount, toID); err != nil {
        return err
    }

    return tx.Commit() // Simpan permanen jika sukses
}
```

Menggunakan `defer tx.Rollback()` untuk memastikan jaring pengaman selalu aktif.

Contoh implementasi kode *best practice* menggunakan ORM:

```go
func TransferAmanGorm(db *gorm.DB, fromID, toID int, amount float64) error {
    // db.Transaction menangani BEGIN, COMMIT, dan ROLLBACK (termasuk saat panic)
    return db.Transaction(func(tx *gorm.DB) error {

        // Gunakan 'tx' (bukan 'db') di dalam blok ini
        if err := tx.Model(&Account{}).Where("id = ?", fromID).
            UpdateColumn("balance", gorm.Expr("balance - ?", amount)).Error; err != nil {
            return err // Otomatis trigger ROLLBACK
        }

        if err := tx.Model(&Account{}).Where("id = ?", toID).
            UpdateColumn("balance", gorm.Expr("balance + ?", amount)).Error; err != nil {
            return err // Otomatis trigger ROLLBACK
        }

        return nil // Otomatis trigger COMMIT
    })
}
```

GORM menyediakan *method* `Transaction`. Jika fungsi *callback* mengembalikan `error`, GORM akan otomatis melakukan `ROLLBACK`. Jika mengembalikan `nil`, otomatis `COMMIT`.

4. Software Quality & Unit Testing

Menulis kode yang "bisa jalan" di mesin lokal saja tidak cukup. Kita perlu membangun kultur di mana kualitas dijaga oleh sistem, bukan sekadar asumsi manual.

Kenapa Perlu Testing?

- **Shift-Left Testing (Deteksi Dini):** Semakin lambat *bug* ditemukan, semakin mahal biaya perbaikannya.
- **Refactoring Confidence:** Dengan Unit Test yang komprehensif, engineer bisa mengoptimalkan logika atau memperbarui library dengan tenang
- **Living Documentation:** Cara terbaik memahami apa yang dilakukan oleh sebuah fungsi dan apa saja parameter masukannya adalah dengan membaca skenario *test*-nya.
- **Regression prevention**: Fitur yang udah beres nggak tiba-tiba rusak saat nambahin fitur baru.
- **Faster debugging**: Kalau test gagal, bisa langsung tahu unit mana yang bermasalah, tanpa perlu cari-cari dari laporan bug.

AAA Pattern

- **Arrange:** Menyiapkan data awal, parameter, dan membuat *mock* (objek tiruan).
- **Act:** Memanggil fungsi utama yang ingin diuji.
- **Assert:** Membandingkan hasil dari *Act* dengan ekspektasi kita.

Setup & Tools

Buat file dengan akhiran _test.go (misal: wallet_test.go),

```
project/
   ├── wallet/
   |   ├──  wallet.go
   |   ├── wallet_test.go
```

Contoh implementasi wallet_test.go:

```go
package wallet

import "testing"

func TestTransferBalance(t *testing.T) {
    // Test logic di sini
    result := Add(2, 3)
    if result != 5 {
        t.Errorf("expected 5, got %d", result)
    }
}
```

Untuk menjalankan, lakukan:

```
go test ./...        # Jalankan semua test di semua package
go test ./wallet/... # Jalankan test di package wallet saja
go test -v ./...     # Verbose: tampilkan nama tiap test yang dijalankan
```

Ada juga library `testify/assert`, yang membuat membuat kode validasi menjadi satu baris daripada menggunakan banyak `if hasil ≠ ekspektasi`, disingkat menjadi `assert.NoError, assert.Equal, assert.Nil`

Contoh penerapan kode menggunakan `testify/assert`:

```go
func TestHitungDiskon(t *testing.T) {
    // 1. ARRANGE
    hargaAwal := 100000
    persenDiskon := 20
    ekspektasiHasil := 80000

    // 2. ACT
    hasilAkhir := HitungDiskon(hargaAwal, persenDiskon)

    // 3. ASSERT
    assert.Equal(t, ekspektasiHasil, hasilAkhir, "Diskon 20% dari 100k harusnya 80k")
}
```

Mocking

Mock adalah "pemeran pengganti" atau *stuntman* untuk dependensi eksternal (Database, API Pihak Ketiga, Redis). Dengan *mocking*, kita memprogram tiruan *database* untuk merespons sesuatu sesuai skenario yang kita inginkan

Kenapa harus di-mock?
Unit test harus berjalan cepat, independen, dan repeatable. Kalau unit test harus connect ke database tiap kali dijalankan, maka:

- **Lambat**: Query ke database nyata bisa makan waktu puluhan hingga ratusan milidetik.
- **Tidak repeatable**: State database bisa berbeda setiap kali test jalan data bisa berubah, database bisa down, koneksi bisa timeout.

5. Unit Test untuk Logic Transaksi

Untuk setiap fungsi yang kita test, kita perlu memikirkan setidaknya dua skenario:

- Happy path (positive test): Semua berjalan sesuai rencana, pastikan state akhirnya benar.
- Error path (negative test): Ada sesuatu yang gagal, pastikan sistem bereaksi dengan benar (rollback, return error yang tepat, nggak ada side effect yang nggak diinginkan).

Menggunakan library `testify/mock`:

Setup interface dan setup mock:

```go
// repository.go
type WalletRepository interface {
    BeginTx() (Transaction, error)
}

type Transaction interface {
    Deduct(userID string, amount int64) error
    Credit(userID string, amount int64) error
    Commit() error
    Rollback() error
}
```

```go
// mock_repository.go (biasanya di-generate otomatis atau ditulis manual)
type MockTransaction struct {
    mock.Mock
}

func (m *MockTransaction) Deduct(userID string, amount int64) error {
    args := m.Called(userID, amount)
    return args.Error(0)
}

func (m *MockTransaction) Credit(userID string, amount int64) error {
    args := m.Called(userID, amount)
    return args.Error(0)
}

func (m *MockTransaction) Commit() error {
    args := m.Called()
    return args.Error(0)
}

func (m *MockTransaction) Rollback() error {
    args := m.Called()
    return args.Error(0)
}
```

Memvalidasi State Sukses (Positive Test):

```go
func TestTransferSuccess(t *testing.T) {
    // Arrange: setup mock
    mockTx := new(MockTransaction)

    // Define ekspektasi: fungsi-fungsi ini akan dipanggil dengan argumen ini,
    // dan akan return nilai ini
    mockTx.On("Deduct", "user-A", int64(100000)).Return(nil)
    mockTx.On("Credit", "user-B", int64(100000)).Return(nil)
    mockTx.On("Commit").Return(nil)

    // Inject mock ke service
    service := NewTransferService(mockTx)

    // Act: jalankan fungsi yang mau kita test
    err := service.Transfer("user-A", "user-B", 100000)

    // Assert: verifikasi hasilnya
    assert.NoError(t, err)                  // Tidak boleh ada error
    mockTx.AssertCalled(t, "Commit")        // Commit HARUS dipanggil
    mockTx.AssertNotCalled(t, "Rollback")   // Rollback TIDAK BOLEH dipanggil
    mockTx.AssertExpectations(t)            // Semua ekspektasi di atas terpenuhi
}
```

Perhatikan apa yang sedang kita verifikasi: bukan cuma bahwa fungsinya return `nil` error, tapi secara eksplisit bahwa `Commit()` dipanggil dan `Rollback()` tidak. Ini adalah behavioral verification. Kita test perilaku kode, bukan cuma outputnya.

Memvalidasi State Gagal (Negative Test):

```go
func TestTransferFails(t *testing.T) {
    // Arrange
    mockTx := new(MockTransaction)

    simulatedError := errors.New("database connection lost")

    mockTx.On("Deduct", "user-A", int64(100000)).Return(nil)
    // Simulasikan: Credit gagal dengan error tertentu
    mockTx.On("Credit", "user-B", int64(100000)).Return(simulatedError)
    mockTx.On("Rollback").Return(nil)

    service := NewTransferService(mockTx)

    // Act
    err := service.Transfer("user-A", "user-B", 100000)

    // Assert
    assert.Error(t, err)                    // Harus return error
    assert.ErrorIs(t, err, simulatedError)  // Dan errornya harus yang tadi
    mockTx.AssertCalled(t, "Rollback")      // Rollback HARUS dipanggil
    mockTx.AssertNotCalled(t, "Commit")     // Commit TIDAK BOLEH dipanggil
    mockTx.AssertExpectations(t)
}
```

Dengan dua test ini, kita sudah memverifikasi dua hal paling krusial dari logic transaksi: bahwa pada kondisi sukses Commit yang jalan, dan pada kondisi gagal Rollback yang jalan. Tanpa harus menyentuh database sekalipun.

Measuring Test Coverage

Test coverage adalah metrik yang mengukur seberapa banyak baris/branch kode kita dieksekusi oleh test suite. ini adalah indikator awal yang berguna untuk menemukan bagian kode yang belum punya safety net testing sama sekali.

Menjalankan Coverage di Go

```
# Generate coverage profile
go test -coverprofile=coverage.out ./...

# Lihat summary di terminal
go tool cover -func=coverage.out

# Buka laporan visual di browser
go tool cover -html=coverage.out
```

Output `cover -func` akan terlihat seperti ini:

```
wallet/wallet.go:Transfer         87.5%
wallet/wallet.go:Validate         100.0%
wallet/service.go:ProcessPayment  62.5%
total:                            (statements) 78.2%
```

Coverage report adalah alat untuk menemukan bagian kode yang belum punya test sama sekali.

Yang jauh lebih penting dari angka coverage adalah kualitas test-nya: apakah assertion-nya bermakna? Apakah test mensimulasikan kondisi-kondisi yang bisa terjadi di production? Apakah negative case dan edge case sudah ter-cover?